Pipeline

Dataset

> **NOTE: DEADLINE TỐI THỨ 4 NGÀY 14/10**

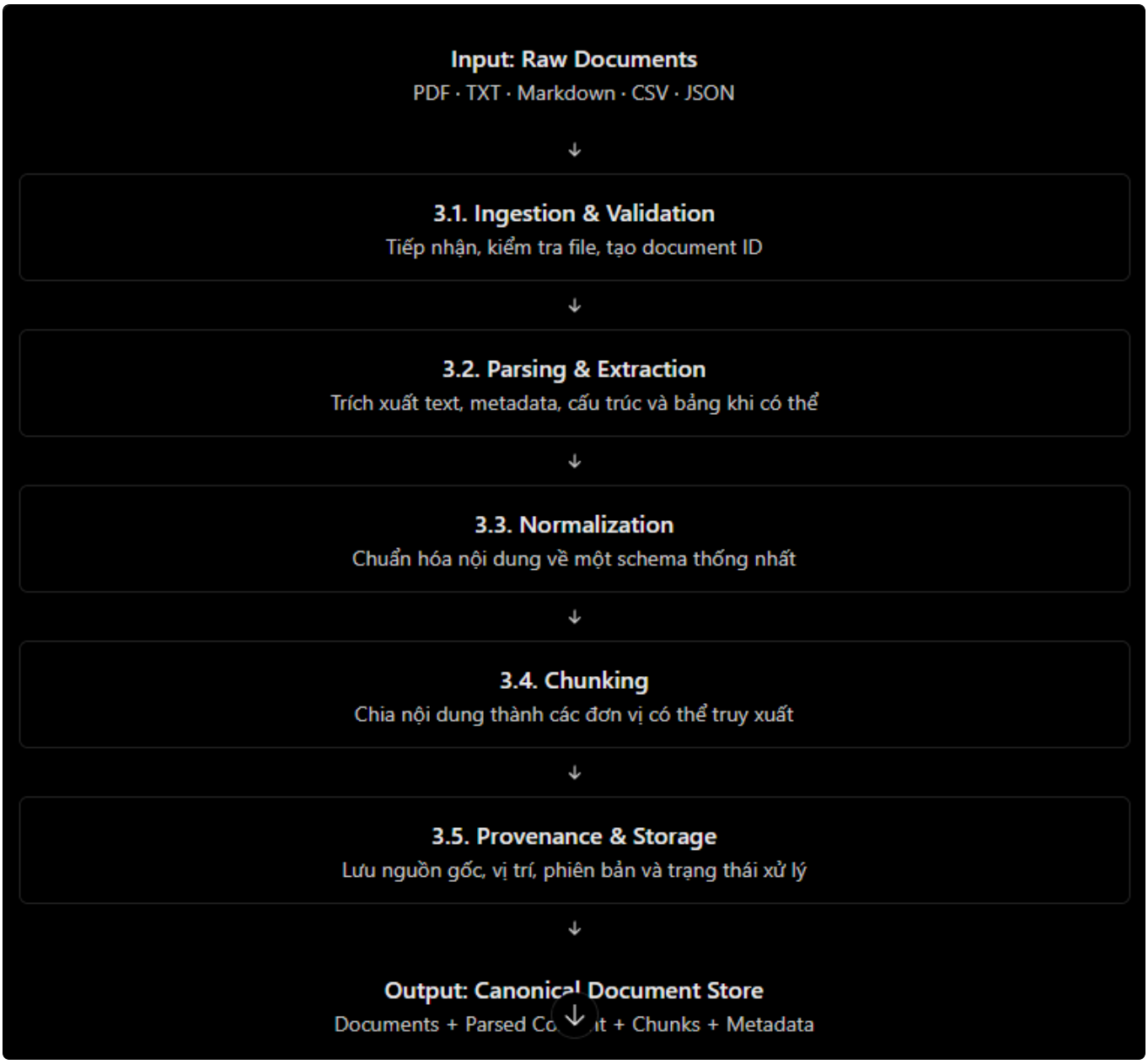
Phân vùng data

- Kiến thức về **danh mục** rủi ro bảo mật và lỗ hỏng: => **Nguyên**
  - Đây là miền kiến thức nền tảng để hệ thống nhận diện chủ đề, phân loại câu hỏi và tìm tài liệu liên quan.
  - Nguồn gợi ý: OWASP Top 10, MITRE CWE
- Kiến thức về **phòng chống, khắc phục** lỗi bảo mật: => **Long, Nguyên**
  - Miền này giúp hệ thống trả lời câu hỏi "Làm thế nào để giảm thiểu hoặc khắc phục lỗ hổng?".
  - Nguồn gợi ý: OWASP Cheat Sheet Series
- Kiến thức về **phương pháp kiểm thử** => **Bảo, Trí**
  - Giúp hiểu cần kiểm tra và xác minh lỗi ra sao.
  - Nguồn gợi ý: OWASP Web Security Testing Guide (WSTG)
- Kiến thức về **tiêu chuẩn, mức độ rủi ro và ngữ cảnh triển khai bảo mật** => **Diệp, Kiệt**
  - Giúp xác định tiêu chuẩn nào áp dụng, mức độ ưu tiên của vấn đề và biện pháp nào phù hợp với bối cảnh cụ thể
  - Nguồn gợi ý: FIRST.org - Common Vulnerability Scoring System, OWASP ASVS
- Bộ **Báo cáo Pentest** mẫu dùng để điều chỉnh kĩ thuật (số lượng vừa 20-25 báo cáo mẫu) => **Kiệt**

Lỗ hỏng chính

- **SQL Injection** => A05:2025 - Injection
- **Cross-Site Scripting** => A05:2025 - Injection
- **Broken Access Control** => A01:2025 - Broken Access Control
- **Broken Authentication / Session Management** => A07:2025 - Authentication Failures
- **Security Misconfiguration** => A02:2025 - Security Misconfiguration

Document Ingestion, Parsing



Data Ingestion

> ***Tiếp nhận, kiểm tra file, tạo document ID***

> **NOTE: DEADLINE TỐI T3 NGÀY 20/10**

1. **Data Ingestion**: => **Long**
   - *Những việc cần làm
     - File discovery: đọc danh sách tài liệu từ thư mục dữ liệu hoặc nguồn input.
     - Format detection: xác định loại file như PDF, TXT, MD, CSV hoặc JSON.
     - Input validation: kiểm tra file tồn tại, không rỗng, có định dạng hỗ trợ và có thể truy cập.
     - Document identification: tạo `document_id` duy nhất cho mỗi tài liệu.

    - Duplicate detection: nhận diện các file trùng nhau, chẳng hạn sử dụng SHA-256 hash.
    - Processing status: theo dõi trạng thái `pending`, `processing`, `success` hoặc `failed`.
    - Error handling: lưu nguyên nhân khi file không hợp lệ hoặc không thể xử lý.
- **Chi tiết**:
    - Xây dựng 2 Schema:
        - Document: cho lưu tài liệu gốc
        - Document_version: theo dõi trạng thái cập nhật của tài liệu (chính trong pipeline)

```
# Document
    {
        document_id
        name
        created_at
    }

# Document_version
    {
        version_id
        document_id
        version_number
        file_name
        file_path
        file_format
        file_size_bytes
        sha256_hash
        status
        created_at
    }
```

2. **Document Parsing => Nguyên, Bảo**

- Những việc cần làm
    - Trích xuất văn bản, số trang và thông tin cấu trúc tài liệu
    - Nhận diện tiêu đề, section, đoạn văn
    - Trích xuất dữ liệu dạng bảng
- ***Schema*** : Content_block

```
    {
        document_id
        version_id
```

```
        parser_type
        block_id
        block_idx
        block_type
        text_content
        structured_content
            header_cols
            rows
        source_locator
            page_number
            bbox
            start_line
            end_line
            table_idx
            start_row
            end_row
        metadata
            heading_path
            heading_level
            code_language
    }
```

- **Cấp khối nội dung (Block Level):**
  - `block_id` : Định danh duy nhất của khối (ví dụ: `blk_001`). PDF
  - `block_idx` : Thứ tự vị trí xuất hiện của khối trong tài liệu (0, 1, 2,...).
  - `block_type` : Phân loại khối (`paragraph` , `heading` , `table` , `code` , `list`). PNG
  - `text_content` : Chuỗi văn bản trích xuất (văn bản thường hoặc chuỗi bảng Markdown). PDF + 1
  - `structured_content` : Dữ liệu bảng có cấu trúc (áp dụng khi `block_type = table` , các trường hợp khác là `null`): PDF + 1
    - `headers` : Danh sách tên các cột (`list[str]`). PDF + 1
    - `rows` : Danh sách các dòng dữ liệu 2D (`list[list[str]]`). PDF + 1

- `source_locator` : Tọa độ vị trí gốc để truy vết bằng chứng (Traceability): PDF + 1
  - `page_number` : Số trang chứa khối (áp dụng cho PDF). PDF + 1
  - `bbox` : Tọa độ hộp bao `[x0, y0, x1, y1]` (áp dụng cho PDF). PNG
  - `start_line` : Dòng bắt đầu trong tệp (áp dụng cho Markdown). PNG
  - `end_line` : Dòng kết thúc trong tệp (áp dụng cho Markdown). PNG
  - `table_index` : Thứ tự bảng trong tệp (áp dụng cho CSV). PNG
  - `start_row` : Dòng bắt đầu của nhóm bản ghi (áp dụng cho CSV). PNG
  - `end_row` : Dòng kết thúc của nhóm bản ghi (áp dụng cho CSV). PNG
- `metadata` : Thông tin ngữ cảnh bổ trợ:
  - `heading_path` : Đường dẫn phân cấp tiêu đề chứa khối. PDF + 2
  - `heading_level` : Cấp bậc tiêu đề (1, 2, 3... nếu `block_type = heading`). PDF
  - `code_language` : Ngôn ngữ của đoạn mã (ví dụ: `sql` , `python` , `http` nếu `block_type = code`).
  - `is_scanned_ocr` : Cờ đánh dấu trang quét dạng ảnh/OCR (`true` / `false`).

3. **Text Cleaning & Normalization => Trí**

- <u>Những việc cần làm</u>
  - Chuẩn hóa khoảng trắng dư thừa.
  - Loại bỏ các dòng trống không cần thiết.
  - Chuẩn hóa xuống dòng.
  - Xử lý ký tự đặc biệt hoặc lỗi trích xuất đã xác định.
  - Giữ nguyên các nội dung có ý nghĩa như dấu câu, công thc, tiêu đề và số liệu.
- Chuẩn hóa nhưng vẫn giữ lại bản text gốc đã trích xuất để có thể vận dụng khi cần
- Những phần mà lặp lại so với bước 2 - Ông hỏi AI thử xem cách xử lý nào là tối ưu nhất (giả sử nếu sau khi normalize mà không thay đổi thì lấy nguồn trên, còn nếu có thay đổi thì xử lý như nào,....)

```
{
    normalizer_version
    normalized_id
    block_id
    block_idx
    block_type
    text_content
    structured_content
        header_cols
```

```
        rows
    source_locator
        page_number
        bbox
        start_line
        end_line
        table_idx
        start_row
        end_row
    metadata
        heading_path
        heading_level
        code_language
    normalization_flags
        is_code_or_payload
        has_modified
        char_count_delta
}
```

- `normalization_flags` :
  - `is_code_or_payload` : Cờ boolean đánh dấu khối chứa mã khai thác/payload nhạy cảm (cấm xóa ký tự lạ, ngoặc nhọn, nháy đơn). PDF
  - `has_modified` : Cờ boolean đánh dấu khối này có nội dung thay đổi sau chuẩn hóa hay không.
  - `char_count_delta` : Độ lệch số lượng ký tự sau khi làm sạch (`len(raw) - len(normalized)`).

4. **Document Chunking => Kiệt**

- Mục tiêu: Chia tài liệu thành các đơn vị nhỏ hơn để có thể lập chỉ mục, truy xuất và cung cấp cho LLM mà không cần đưa toàn bộ tài liệu vào ngữ cảnh.
- Phương pháp:
  - Fixed-size chunking: Baseline
  - Structure-aware chunking: Cải tiến

```
{
    chunk_id
    chunk_type
    chunk_idx
    content
    token_count
```

```
    chunk_strategy
    source_block_ids
    source_locator
        page_range
        bbox_list
        line_range
        row_range
        char_offsets
            start_char
            end_char
    structural_context
        heading_path
        has_table
        has_code
}
```

- **Giải thích các thuộc tính:**
  - `chunk_id` : Định danh duy nhất của chunk (ví dụ: `chk_ver8f4a_001`).
  - `content` : Chuỗi văn bản hoàn chỉnh của chunk (đã bổ sung tiền tố heading nếu dùng chiến lược cắt theo cấu trúc). PDF
  - `token_count` : Số lượng token tính theo tokenizer (dùng để kiểm soát tràn ngữ cảnh). PDF + 1
  - `chunk_strategy` : Chiến lược phân đoạn (`fixed_size` hoặc `structure_aware`) để thực hiện thực nghiệm so sánh cải tiến (R8). PDF + 1
  - `source_block_ids` : Danh sách các `block_id` từ Bước 3 cấu thành nên chunk này (ví dụ: `["blk_001", "blk_002"]`).
  - `source_locator` : Vị trí vật lý tổng hợp của chunk trên tài liệu nguồn: PDF + 1
    - `page_range` : Cặp mảng `[start_page, end_page]` (với PDF). PDF
    - `bbox_list` : Danh sách các hộp bao tọa độ để giao diện vẽ khung highlight. PDF + 1
    - `line_range` : Cặp mảng `[start_line, end_line]` (với Markdown). PNG
    - `row_range` : Cặp mảng `[start_row, end_row]` (với CSV). PNG
    - `char_offsets` : Vị trí ký tự bắt đầu (`start_char`) và kết thúc (`end_char`) trong tài liệu đã chuẩn hóa. PDF
  - `structural_context` :
    - `heading_path` : Đường dẫn tiêu đề bao bọc chunk này. PDF + 1
    - `has_table` : Cờ boolean đánh dấu chunk có chứa dữ liệu bảng biểu. PDF
    - `has_code` : Cờ boolean đánh dấu chunk có chứa code/payload. PDF

5. **Metadata Extraction** => **Diệp** => Không xuất ra JSON nữa, convert sang SQLite

- Bổ sung các trường thông tin để đưa các tài liệu về 1 schema thống nhất
- 2 Metadata chính:
  - Document Metadata
  - Chunk Metadata

```
# Document Metadata
    {
        document_id
        title
        document_format
        domain_category
```

```
        total_pages
    }

# Chunk Metadata
    {
        chunk_id
        chunk_idx
        chunk_strategy
        heading_path
        page_number
        line_range
        security_entities (Gen lại AI hỏi thử)
            owasp_category
            vulnerability_type
            cwe_ids
            wstg_ids
            asvs_levels
            severity
        content_flags
            has_code
            has_table
            has_poc
        token_count
    }
```

- `heading_path` : Đường dẫn phân cấp tiêu đề chứa chunk (breadcrumbs, ví dụ: `A01:2025 Broken Access Control > Remediation`). PDF + 2
- `page_number` : Trang vật lý của chunk (nếu là PDF) hoặc `page_range: [start, end]` . PDF + 2
- `line_range` : Dòng bắt đầu và kết thúc (nếu là Markdown). PNG
- `security_entities` : Các thực thể an toàn thông tin được trích xuất bằng biểu thức chính quy (Regex):
  - `owasp_category` : Nhóm rủi ro OWASP liên quan (ví dụ: `A01:2025` , `A05:2025`). PDF
  - `cwe_ids` : Danh sách mã CWE xuất hiện trong chunk (ví dụ: `["CWE-89"]`). PDF
  - `wstg_ids` : Danh sách mã kiểm thử xuất hiện trong chunk (ví dụ: `["WSTG-INPV-05"]`). PDF
  - `asvs_levels` : Cấp độ ASVS nếu có (ví dụ: `["Level 1"]` , `["Level 2"]`). PDF
  - `severity` : Mức độ rủi ro nếu có (`CRITICAL` , `HIGH` , `MEDIUM` , `LOW` , `INFO` , `null`).
  - `cvss_base_score` : Điểm số CVSS nếu đoạn đó nhắc đến chỉ số đánh giá lỗ hổng cụ thể.
- `content_flags` : Cờ phân loại nội dung:
  - `has_code` : Có chứa mã nguồn hoặc lệnh HTTP/bash hay không. PDF + 1
  - `has_table` : Có chứa dữ liệu bảng biểu hay không. PDF + 1
  - `has_poc` : Có chứa payload khai thác thực tế hay không. PDF
- `token_count` : Số lượng token của chunk (tính theo tokenizer). PDF + 1

6. **Provenance & Traceability => Kiệt**
   - Mục tiêu: Đảm bảo rằng khi hệ thống tìm thấy một đoạn văn và đưa ra câu trả lời, nó có thể xác định đoạn đó đến từ tài liệu nào, trang nào và vị trí nào nếu thông tin vị trí đã được lưu.

> *Note*
> **- Mấy cái Schema thì đó là tui tìm hiểu sơ bộ, ae xem xem có áp dụng vô được không (tại**

> **tui cũng ko), hoặc lúc làm ae gen AI hỏi nó thử xem có gợi ý cái nào không "quá" cần thiết có thể bỏ thì bỏ, cái nào lặp lại nhiều hoặc có thể tối ưu gì đấy thì có thể chỉnh sửa**